

PROYECTO DE CURSADA

Sistema de Gestión de Clínica Odontológica "Sonrisa Feliz"

Materia: 3.4.208 Paradigma Orientado a Objetos (Presencial)

Ciclo Lectivo 2026

Documento General del Proyecto – Versión 1.0

1. Introducción

La clínica odontológica "Sonrisa Feliz" creció desde un pequeño consultorio de barrio hasta convertirse en un centro de atención con múltiples profesionales y un flujo constante de pacientes. Su identidad siempre fue la calidez humana: saludar por el nombre, recordar historias clínicas, recomendar turnos de seguimiento. Esa cultura fue suficiente durante años... hasta que el volumen de pacientes y profesionales superó la capacidad de gestión manual.

La libreta de papel en recepción —la misma que alguna vez fue orgullo de orden— empezó a revelar su fragilidad: citas superpuestas, teléfonos mal anotados, confirmaciones que se perdían, reportes que nunca llegaban. Los odontólogos pedían agenda con urgencia; los pacientes reclamaban claridad; la administración necesitaba métricas para decidir. Y la libreta no daba más.

El director de la clínica nos convocó con una petición concreta: "Necesitamos un sistema que funcione y que nos acompañe a crecer. No busquen el ideal imposible: queremos avances reales y visibles en cada etapa". Con esa premisa, definimos un camino iterativo, pedagógico y profesional para construir la solución.

1.1 Contexto del proyecto de cursada

Este proyecto no es solo una práctica de programación; es un viaje de diseño y entrega de valor. El equipo de estudiantes-desarrolladores vivirá la evolución de un sistema de gestión construido enteramente con Java orientado a objetos puro, transitando conscientemente por decisiones de diseño, patrones y buenas prácticas de la industria del software.

A lo largo de cuatro entregas progresivas, el equipo construirá desde el modelo de dominio básico hasta una aplicación funcional con interfaz gráfica Swing, pasando por capas de servicio con patrones GRASP y SOLID, manejo robusto de excepciones, persistencia en archivos y uso avanzado de colecciones. Cada entrega es un hito de esta historia. Cada decisión técnica tiene un porqué.

1.2 Alcance

El sistema abarcará la gestión integral de pacientes, odontólogos y turnos de la clínica. Se implementará utilizando exclusivamente Java SE (Standard Edition), sin frameworks externos, lo que permite a los estudiantes comprender en profundidad los mecanismos fundamentales de la programación orientada a objetos antes de avanzar en su formación hacia tecnologías de mayor nivel de abstracción.

1.3 Objetivos pedagógicos

- Aplicar los pilares de la Programación Orientada a Objetos: clases, objetos, encapsulamiento, herencia y polimorfismo.
- Diseñar y documentar modelos de dominio utilizando UML (diagramas de clases, secuencia y casos de uso).
- Implementar patrones de diseño GRASP (Controller, Expert, Creator, Low Coupling, High Cohesion).
- Aplicar principios SOLID, especialmente SRP (Single Responsibility) y OCP (Open/Closed).
- Dominar el uso de colecciones de Java (List, Map, Set) para la gestión de datos en memoria.

- Diseñar y utilizar excepciones personalizadas para un manejo robusto de errores.
- Persistir datos mediante archivos (CSV o serialización Java).
- Desarrollar interfaces gráficas de usuario con Swing (JFrame, JPanel, JTable, eventos).
- Introducir conceptos de concurrencia mediante hilos (Thread) para tareas asíncronas.

2. Objetivos del Proyecto

2.1 Objetivo general

Desarrollar un sistema de gestión para la clínica odontológica "Sonrisa Feliz" utilizando Java orientado a objetos puro, que permita administrar pacientes (con su domicilio asociado), odontólogos y turnos de manera eficiente, aplicando buenas prácticas de diseño y arquitectura de software.

2.2 Objetivos específicos

- **Modelado de dominio:** Diseñar las entidades del negocio (Paciente, Domicilio, Odontólogo, Turno) con sus atributos, métodos y relaciones, documentados mediante diagramas UML de clases.
- **Encapsulamiento y abstracción:** Implementar clases con visibilidad adecuada, constructores, getters/setters y métodos de negocio, respetando el principio de ocultamiento de información.
- **Herencia y polimorfismo:** Identificar oportunidades de generalización/especialización en el dominio e implementar jerarquías de clases con comportamiento polimórfico.
- **Patrones GRASP y SOLID:** Diseñar la capa de servicios aplicando patrones de asignación de responsabilidades y principios de diseño orientado a objetos.
- **Colecciones Java:** Utilizar List, Map y Set para gestionar repositorios en memoria con operaciones de búsqueda, filtrado y ordenamiento.
- **Excepciones personalizadas:** Diseñar una jerarquía de excepciones del dominio para manejar situaciones de error de forma clara y robusta.
- **Persistencia en archivos:** Implementar lectura y escritura de datos en archivos de texto (CSV) o mediante serialización Java para garantizar la permanencia de la información entre ejecuciones.
- **Interfaz gráfica:** Construir una GUI funcional con Swing que permita realizar todas las operaciones del sistema de forma visual e intuitiva.
- **Concurrencia (opcional):** Aplicar hilos (Thread/Runnable) para tareas asíncronas como recordatorios de turnos o búsquedas en segundo plano.

3. Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades que el sistema debe ofrecer. Se derivan del relevamiento realizado con recepción, administración y dirección de la clínica.

RF1 – Gestión de Pacientes

El sistema debe permitir registrar, consultar, modificar y eliminar pacientes. Cada paciente tiene asociado un domicilio.

Atributo	Tipo	Descripción
id	Long	Identificador único del paciente (autogenerado)
nombre	String	Nombre del paciente
apellido	String	Apellido del paciente
dni	String	Documento Nacional de Identidad (único, no repetible)
email	String	Correo electrónico para notificaciones
fechaIngreso	LocalDate	Fecha de alta en la clínica
domicilio	Domicilio	Domicilio asociado (relación de composición/agregación)

Entidad Domicilio (asociada a Paciente):

Atributo	Tipo	Descripción
id	Long	Identificador único del domicilio
calle	String	Nombre de la calle
numero	String	Número de puerta
localidad	String	Localidad o ciudad
provincia	String	Provincia

Operaciones requeridas:

- Alta de paciente con su domicilio asociado.
- Búsqueda de paciente por ID.
- Búsqueda de paciente por DNI.
- Listado de todos los pacientes.
- Modificación de datos del paciente (incluido su domicilio).
- Eliminación de paciente (validando que no tenga turnos futuros asignados).

RF2 – Gestión de Odontólogos

El sistema debe permitir registrar, consultar, modificar y eliminar odontólogos.

Atributo	Tipo	Descripción
id	Long	Identificador único del odontólogo (autogenerado)
nombre	String	Nombre del profesional
apellido	String	Apellido del profesional
matricula	String	Número de matrícula profesional (único, clave de auditoría)

Operaciones requeridas:

- Alta de odontólogo.
- Búsqueda de odontólogo por ID.
- Búsqueda de odontólogo por matrícula.
- Listado de todos los odontólogos.
- Modificación de datos del odontólogo.
- Eliminación de odontólogo (validando que no tenga turnos futuros asignados).

RF3 – Gestión de Turnos

El sistema debe permitir reservar, consultar, modificar y cancelar turnos, vinculando un paciente con un odontólogo en una fecha y hora determinadas.

Atributo	Tipo	Descripción
id	Long	Identificador único del turno (autogenerado)
paciente	Paciente	Paciente asociado al turno (relación)
odontologo	Odontólogo	Odontólogo que atiende (relación)
fecha	LocalDate	Fecha del turno
hora	LocalTime	Hora del turno
estado	EstadoTurno	Estado: PENDIENTE, CONFIRMADO, CANCELADO, COMPLETADO

Operaciones requeridas:

- Reservar turno (validando disponibilidad del odontólogo en esa fecha/hora).
- Confirmar turno.

- Cancelar turno.
- Modificar turno (cambio de fecha/hora u odontólogo).
- Listar todos los turnos.
- Buscar turno por ID.

RF4 – Búsquedas y Reportes

El sistema debe ofrecer las siguientes consultas y reportes:

- Listar todos los pacientes ordenados alfabéticamente.
- Buscar paciente por DNI.
- Listar todos los turnos de un paciente determinado.
- Listar todos los turnos de un odontólogo determinado.
- Listar turnos por fecha (agenda del día).
- Listar turnos por estado (pendientes, cancelados, completados).
- (Opcional) Reporte de pacientes nuevos en un período dado.
- (Opcional) Agenda semanal por odontólogo.

RF5 – Interfaz Gráfica (Entrega 4)

En la fase final del proyecto, todas las operaciones anteriores deberán ser accesibles a través de una interfaz gráfica de usuario (GUI) construida con Swing. La GUI debe incluir ventanas para la gestión de pacientes, odontólogos y turnos, así como pantallas de búsqueda y reportes.

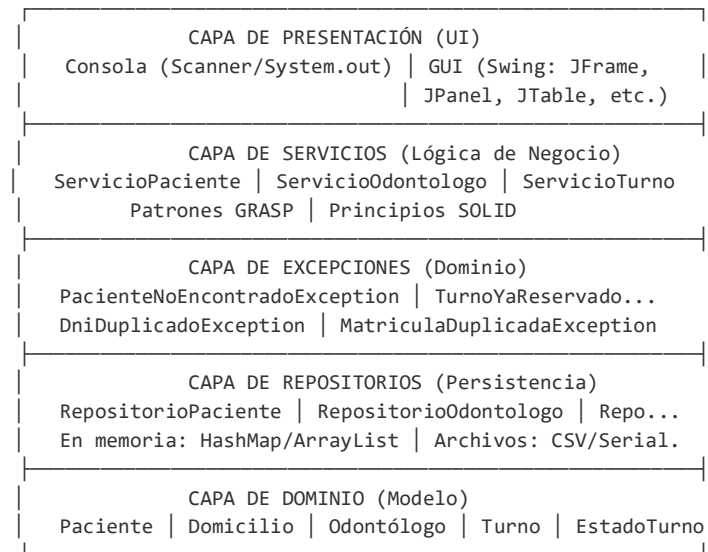
4. Requerimientos No Funcionales

ID	Requerimiento	Descripción
RNF1	Código limpio y documentado	El código debe seguir convenciones Java estándar (camelCase, PascalCase) y estar documentado con Javadoc en clases y métodos públicos.
RNF2	Patrones GRASP	Se deben aplicar los patrones Controller, Information Expert, Creator, Low Coupling y High Cohesion en el diseño de las capas de servicio.
RNF3	Principios SOLID	Especialmente SRP (cada clase una responsabilidad) y OCP (abierto a extensión, cerrado a modificación). Justificar las decisiones en los informes.
RNF4	Manejo robusto de errores	Excepciones personalizadas del dominio. Ningún error debe producir un crash silencioso; todos los errores deben ser capturados y comunicados al usuario.
RNF5	Persistencia en archivos	Los datos de pacientes, odontólogos y turnos deben persistir entre ejecuciones mediante archivos CSV o serialización Java.
RNF6	Interfaz intuitiva	Consola interactiva con menús claros (entregas 1 a 3). GUI Swing funcional e intuitiva (entrega 4).
RNF7	Versionado de código	Se recomienda el uso de Git para control de versiones. Commits frecuentes con mensajes descriptivos.

5. Arquitectura del Sistema

El sistema se organiza en una arquitectura en capas sin frameworks externos. Cada capa tiene una responsabilidad claramente definida y se comunica únicamente con las capas adyacentes, promoviendo el bajo acoplamiento y la alta cohesión.

5.1 Diagrama de capas



5.2 Descripción de cada capa

Capa de Dominio (modelo)

Contiene las entidades del negocio: Paciente, Domicilio, Odontólogo, Turno y el enumerado EstadoTurno. Son clases Java puras (POJOs) con atributos privados, constructores, getters/setters y métodos de negocio. Representan los conceptos centrales del dominio de la clínica y no tienen dependencia con ninguna otra capa.

Paquete sugerido: modelo (o dominio)

Capa de Repositorios (persistencia)

Responsable de almacenar y recuperar las entidades. Se implementa en dos niveles progresivos:

- Repositorios en memoria: utilizan colecciones Java (HashMap<Long, Paciente>, ArrayList<Turno>, etc.) para almacenar datos durante la ejecución.
- Persistencia en archivos (a partir de la Entrega 3): lectura y escritura de archivos CSV o serialización Java (ObjectOutputStream/ObjectInputStream) para que los datos sobrevivan entre ejecuciones.

Se recomienda definir una interfaz IRepositoryo<T> con los métodos CRUD básicos (guardar, buscarPorId, buscarTodos, actualizar, eliminar) e implementarla para cada entidad. Esto facilita cambiar la estrategia de persistencia sin modificar las capas superiores (OCP).

Paquete sugerido: repositorio

Capa de Servicios (lógica de negocio)

Orquesta las operaciones del sistema, aplicando reglas de negocio y validaciones. Cada servicio utiliza uno o más repositorios y lanza excepciones personalizadas ante situaciones de error.

- ServicioPaciente: gestiona el CRUD de pacientes con validaciones (DNI no duplicado, existencia del paciente, etc.).
- ServicioOdontologo: gestiona el CRUD de odontólogos con validaciones (matrícula no duplicada, etc.).
- ServicioTurno: reserva, cancela, modifica y consulta turnos validando disponibilidad, existencia de paciente y odontólogo, coherencia de fechas, etc.

Los servicios aplican los patrones GRASP: Controller (orquestan el flujo), Expert (delegan a quien tiene la información), Creator (crean objetos donde corresponde). Además, cumplen principios SOLID: cada servicio tiene una única responsabilidad (SRP) y está abierto a extensión (OCP) mediante interfaces.

Paquete sugerido: servicio

Capa de Excepciones (dominio)

Contiene excepciones personalizadas que representan situaciones de error específicas del negocio. Todas extienden de RuntimeException o Exception según la semántica deseada.

- PacienteNoEncontradoException: paciente no existe en el sistema.
- OdontologoNoEncontradoException: odontólogo no existe en el sistema.
- TurnoYaReservadoException: el odontólogo ya tiene un turno en esa fecha/hora.
- DniDuplicadoException: ya existe un paciente con ese DNI.
- MatriculaDuplicadaException: ya existe un odontólogo con esa matrícula.
- TurnoNoEncontradoException: el turno buscado no existe.

Paquete sugerido: excepcion

Capa de Presentación (UI)

Es la interfaz con el usuario. Se implementa en dos fases:

- Entregas 1 a 3 – Interfaz por consola: Menús interactivos con Scanner para capturar entrada del usuario y System.out.println() para mostrar resultados. Clara, funcional y navegable.
- Entrega 4 – Interfaz gráfica con Swing: JFrame como ventana principal, JPanel para organizar secciones, JTable para listados, JTextField/JComboBox para formularios, JButton con ActionListener para acciones. Integración completa con la capa de servicios.

Paquete sugerido: presentacion (con subpaquetes consola y gui si se desea)

5.3 Estructura de paquetes sugerida

```
com.clinica.sonrisafeliz
├── modelo/
│   ├── Paciente.java
```

```
|
|   ├── Domicilio.java
|   ├── Odontologo.java
|   ├── Turno.java
|   └── EstadoTurno.java (enum)
├── repositorio/
|   ├── IRepository.java (interfaz genérica)
|   ├── RepositorioPaciente.java
|   ├── RepositorioOdontologo.java
|   └── RepositorioTurno.java
├── servicio/
|   ├── ServicioPaciente.java
|   ├── ServicioOdontologo.java
|   └── ServicioTurno.java
├── excepcion/
|   ├── PacienteNoEncontradoException.java
|   ├── OdontologoNoEncontradoException.java
|   ├── TurnoYaReservadoException.java
|   ├── DniDuplicadoException.java
|   └── MatriculaDuplicadaException.java
├── presentacion/
|   ├── consola/
|   │   └── MenuPrincipal.java
|   └── gui/
|       ├── VentanaPrincipal.java
|       ├── PanelPacientes.java
|       ├── PanelOdontologos.java
|       └── PanelTurnos.java
└── Main.java
```

6. Modelo de Dominio

El modelo de dominio representa las entidades centrales del negocio de la clínica y sus relaciones. A continuación se describe cada entidad en detalle.

6.1 Paciente

Entidad central del sistema. Representa a una persona que se atiende en la clínica. Se asocia a un Domicilio (relación de composición: el domicilio no tiene sentido independiente fuera del paciente en este sistema) y puede tener múltiples turnos asignados.

Atributos: id (Long), nombre (String), apellido (String), dni (String), email (String), fechaIngreso (LocalDate), domicilio (Domicilio).

Métodos sugeridos: toString(), equals()/hashCode() por DNI, getNombreCompleto().

6.2 Domicilio

Clase que modela la dirección de un paciente. En el contexto de este sistema, Domicilio es una clase de soporte asociada a Paciente. No tiene flujo de gestión autónomo (no se administran domicilios independientemente).

Atributos: id (Long), calle (String), numero (String), localidad (String), provincia (String).

Métodos sugeridos: toString() con formato legible ("calle número, localidad, provincia").

6.3 Odontólogo

Entidad central que representa a un profesional odontológico habilitado para atender pacientes. La matrícula es un dato clave de auditoría y habilitación profesional.

Atributos: id (Long), nombre (String), apellido (String), matricula (String).

Métodos sugeridos: toString(), equals()/hashCode() por matrícula, getNombreCompleto().

Nota sobre herencia (opcional, Entrega 2): si el equipo identifica especialidades diferenciadas (por ejemplo, Ortodoncista, Endodoncista, Cirujano), puede modelar una jerarquía donde Odontólogo sea la clase base y las especialidades sean subclases con atributos o comportamientos adicionales. Esto es un excelente ejercicio de herencia y polimorfismo.

6.4 Turno

Entidad que vincula un paciente con un odontólogo en una fecha y hora determinadas. Tiene un estado que modela su ciclo de vida (PENDIENTE → CONFIRMADO → COMPLETADO, o PENDIENTE → CANCELADO).

Atributos: id (Long), paciente (Paciente), odontologo (Odontólogo), fecha (LocalDate), hora (LocalTime), estado (EstadoTurno).

EstadoTurno (enum): PENDIENTE, CONFIRMADO, CANCELADO, COMPLETADO.

Métodos sugeridos: toString(), esFuturo(), estaDisponible().

6.5 Relaciones entre entidades

Relación	Tipo	Cardinalidad	Descripción
Paciente → Domicilio	Composición	1:1	Todo paciente tiene un domicilio. El domicilio no existe sin el paciente.
Turno → Paciente	Asociación	N:1	Un turno pertenece a un paciente. Un paciente puede tener muchos turnos.
Turno → Odontólogo	Asociación	N:1	Un turno es atendido por un odontólogo. Un odontólogo puede tener muchos turnos.

Diagrama UML de clases requerido: el equipo debe elaborar un diagrama UML de clases que incluya las cuatro entidades con sus atributos, métodos, relaciones (asociación, composición), cardinalidades y navegabilidades. Se sugiere utilizar herramientas como StarUML, Draw.io o Lucidchart.

7. Casos de Uso Principales

Los casos de uso describen las interacciones entre los actores (usuarios del sistema) y el sistema. El actor principal es el Recepcionista/Administrador de la clínica.

CU1 – Registrar Paciente

Elemento	Descripción
Actor	Recepcionista
Precondiciones	El sistema está en ejecución.
Flujo principal	1. El usuario selecciona "Registrar paciente". 2. Ingresa nombre, apellido, DNI, email. 3. Ingresa datos de domicilio (calle, número, localidad, provincia). 4. El sistema valida que el DNI no esté registrado. 5. El sistema genera un ID y registra al paciente con fecha de ingreso actual. 6. El sistema muestra confirmación.
Flujo alternativo	4a. Si el DNI ya existe, el sistema lanza <code>DniDuplicadoException</code> y muestra mensaje de error.
Postcondiciones	El paciente queda registrado en el sistema con su domicilio.

CU2 – Registrar Odontólogo

Elemento	Descripción
Actor	Administrador
Precondiciones	El sistema está en ejecución.
Flujo principal	1. El usuario selecciona "Registrar odontólogo". 2. Ingresa nombre, apellido, matrícula. 3. El sistema valida que la matrícula no esté registrada. 4. El sistema genera un ID y registra al odontólogo. 5. El sistema muestra confirmación.
Flujo alternativo	3a. Si la matrícula ya existe, el sistema lanza <code>MatriculaDuplicadaException</code> .
Postcondiciones	El odontólogo queda registrado y habilitado para recibir turnos.

CU3 – Reservar Turno

Elemento	Descripción
Actor	Recepcionista
Precondiciones	Existe al menos un paciente y un odontólogo registrados.

Flujo principal	1. El usuario selecciona "Reservar turno". 2. Busca y selecciona un paciente (por DNI o listado). 3. Busca y selecciona un odontólogo (por matrícula o listado). 4. Ingresa fecha y hora deseadas. 5. El sistema valida disponibilidad del odontólogo en esa fecha/hora. 6. El sistema crea el turno con estado PENDIENTE. 7. El sistema muestra confirmación con los datos del turno.
Flujo alternativo	5a. Si el odontólogo ya tiene turno en esa fecha/hora, lanza TurnoYaReservadoException. 2a/3a. Si el paciente u odontólogo no existe, lanza excepción correspondiente.
Postcondiciones	El turno queda registrado con estado PENDIENTE.

CU4 – Buscar Paciente por DNI

Elemento	Descripción
Actor	Recepcionista
Precondiciones	El sistema está en ejecución.
Flujo principal	1. El usuario selecciona "Buscar paciente por DNI". 2. Ingresa el DNI. 3. El sistema busca en el repositorio. 4. Muestra los datos completos del paciente (incluido domicilio).
Flujo alternativo	3a. Si no existe paciente con ese DNI, lanza PacienteNoEncontradoException.
Postcondiciones	Se muestra la información del paciente.

CU5 – Listar Turnos de un Paciente

Elemento	Descripción
Actor	Recepcionista
Precondiciones	Existe el paciente en el sistema.
Flujo principal	1. El usuario selecciona "Listar turnos de paciente". 2. Busca y selecciona un paciente. 3. El sistema filtra los turnos asociados al paciente. 4. Muestra listado con fecha, hora, odontólogo y estado de cada turno.
Flujo alternativo	3a. Si el paciente no tiene turnos, muestra mensaje informativo.
Postcondiciones	Se muestra el listado de turnos del paciente.

Diagrama UML de Casos de Uso requerido: el equipo debe elaborar un diagrama que muestre los actores (Recepcionista, Administrador) y los casos de uso principales con sus relaciones.

8. Diagramas UML Requeridos

La documentación UML es parte integral del proyecto. Los diagramas deben reflejar el diseño actual del sistema y actualizarse en cada entrega.

Diagrama	Contenido	Entrega
Diagrama de Clases	Todas las entidades con atributos, métodos, relaciones (asociación, composición), cardinalidades y navegabilidades. Incluir clases de servicio y repositorio.	Entrega 1 (inicial) y actualizar en cada entrega
Diagrama de Casos de Uso	Actores (Recepcionista, Administrador) y todos los casos de uso principales con relaciones <<include>> y <<extend>> si aplica.	Entrega 1 o 2
Diagrama de Secuencia	Al menos 2 diagramas: uno para "Reservar Turno" y otro para "Buscar Paciente por DNI". Mostrar interacción entre UI → Servicio → Repositorio.	Entrega 3
Diagrama de Paquetes (opcional)	Organización del sistema en capas/paquetes, mostrando dependencias entre ellos.	Entrega 3 o 4

Herramientas sugeridas: StarUML, Draw.io (diagrams.net), Lucidchart, Visual Paradigm Community Edition.

9. Entregas del Proyecto

El proyecto se desarrolla de forma incremental a lo largo de la cursada, organizado en cuatro entregas progresivas. Cada entrega suma funcionalidad y complejidad técnica sobre la anterior, reflejando una evolución profesional del sistema. Las fechas están alineadas con el cronograma de la materia.

ENTREGA 1 — Modelo de Dominio y UML Básico

Fecha de entrega: Clase 5 – Lunes 14 de abril de 2026

Objetivo

Construir la base del sistema: las clases del modelo de dominio con sus atributos, constructores, métodos y relaciones, acompañadas de un diagrama UML de clases. El sistema debe compilar y ejecutarse con un main de prueba que demuestre la creación de objetos y sus relaciones.

Entregables

- Diagrama UML de clases completo (Paciente, Domicilio, Odontólogo, Turno, EstadoTurno) con atributos, métodos, relaciones y cardinalidades.
- Implementación de las clases de dominio en Java: atributos privados, constructores (vacío y parametrizado), getters/setters, toString().
- Relación Paciente–Domicilio implementada como atributo de asociación (composición).
- Relación Turno–Paciente y Turno–Odontólogo implementadas.
- Enum EstadoTurno con los valores PENDIENTE, CONFIRMADO, CANCELADO, COMPLETADO.
- Clase Main con código de prueba: crear pacientes con domicilio, odontólogos y turnos; imprimir por consola.
- Código compilable y ejecutable sin errores.

Conceptos evaluados

Clases y objetos, encapsulamiento, constructores, relaciones entre clases (asociación, composición), enumerados, UML de clases.

Formato de entrega

- Código fuente del proyecto Java (carpeta src).
- Archivo PDF con el diagrama UML de clases.
- Informe breve (1–2 páginas): decisiones de diseño, justificación de relaciones, reflexión sobre encapsulamiento.

ENTREGA 2 — Servicios, Herencia, Polimorfismo y Patrones

Fecha de entrega: Clase 8 – Lunes 5 de mayo de 2026

Objetivo

Incorporar la capa de servicios y repositorios en memoria, aplicando patrones GRASP y principios SOLID. Agregar herencia y polimorfismo al modelo si se identifican oportunidades de generalización. El sistema debe tener una interfaz por consola funcional para realizar operaciones CRUD.

Entregables

- Capa de servicios implementada: ServicioPaciente, ServicioOdontologo, ServicioTurno con lógica de negocio y validaciones básicas.
- Capa de repositorios en memoria: RepositorioPaciente (HashMap<Long, Paciente>), RepositorioOdontologo (HashMap<Long, Odontologo>), RepositorioTurno (ArrayList<Turno> o HashMap).
- Interfaz IRepositoryo<T> con métodos CRUD genéricos.
- Aplicación de patrones GRASP: Controller (servicios orquestan), Expert (la entidad que tiene la información responde), Creator (quien tiene los datos crea el objeto).
- Aplicación de principios SOLID: SRP (cada clase una responsabilidad), OCP (extensible vía interfaces).
- (Opcional) Herencia: por ejemplo, jerarquía de Odontólogo con especialidades (Ortodoncista, Endodoncista) o jerarquía de Turno (TurnoUrgente, TurnoControl).
- (Opcional) Polimorfismo: comportamiento diferenciado según el tipo (ej: calcularDuracion() según especialidad).
- Interfaz por consola funcional con menú principal y submenús para Pacientes, Odontólogos y Turnos.
- Operaciones CRUD completas accesibles desde consola.

Conceptos evaluados

Herencia, polimorfismo, interfaces, colecciones (HashMap, ArrayList), patrones GRASP, principios SOLID, arquitectura en capas.

Formato de entrega

- Código fuente del proyecto Java actualizado.
- Archivo PDF con diagrama UML de clases actualizado (incluyendo servicios, repositorios, interfaces).
- Informe (2–3 páginas): patrones GRASP aplicados (con ejemplos concretos), principios SOLID demostrados, decisiones de herencia/polimorfismo.

ENTREGA 3 — Excepciones, Colecciones Avanzadas y Persistencia

Fecha de entrega: Clase 14 – Lunes 16 de junio de 2026

Objetivo

Robustecer el sistema con excepciones personalizadas, uso avanzado de colecciones para búsquedas y filtros, y persistencia de datos en archivos. El sistema debe poder guardar y cargar datos automáticamente al iniciar y cerrar la aplicación.

Entregables

- Jerarquía de excepciones personalizadas: `PacienteNoEncontradoException`, `OdontologoNoEncontradoException`, `TurnoYaReservadoException`, `DniDuplicadoException`, `MatriculaDuplicadaException`, `TurnoNoEncontradoException`.
- Manejo robusto de errores en todos los servicios: try-catch-finally donde corresponda, mensajes claros al usuario.
- Uso avanzado de colecciones: búsquedas filtradas (pacientes por apellido, turnos por fecha), ordenamientos (`Comparable/Comparator`), iteración con for-each y/o iteradores.
- Persistencia en archivos implementada para las tres entidades (Pacientes, Odontólogos, Turnos).
- Opción A – Archivos CSV: lectura y escritura con `BufferedReader/BufferedWriter`, parseo de líneas.
- Opción B – Serialización Java: implementar `Serializable` en las entidades, usar `ObjectOutputStream/ObjectInputStream`.
- Carga automática de datos al iniciar la aplicación y guardado al cerrar (o bajo demanda desde el menú).
- Al menos 2 diagramas de secuencia UML: "Reservar Turno" y "Buscar Paciente por DNI".
- Interfaz por consola mejorada con manejo de errores visible para el usuario.

Conceptos evaluados

Excepciones (checked/unchecked, personalizadas, jerarquía), entrada/salida (`java.io`), colecciones avanzadas (búsquedas, filtros, ordenamiento), diagramas de secuencia.

Formato de entrega

- Código fuente del proyecto Java actualizado.
- Archivos de datos generados (CSV o archivos serializados).
- Archivo PDF con diagramas UML actualizados (clases + secuencia).
- Informe (3–4 páginas): excepciones diseñadas y justificación, estrategia de persistencia elegida, uso de colecciones avanzadas.

ENTREGA 4 — Interfaz Gráfica (Swing), Eventos e Hilos

Fecha de entrega: Clase 16 – Lunes 30 de junio de 2026

Objetivo

Reemplazar la interfaz de consola por una GUI completa con Swing, implementar manejo de eventos e integrar opcionalmente hilos para tareas asíncronas. El sistema debe ser funcional de punta a punta: desde la interfaz gráfica hasta la persistencia en archivos.

Entregables

- Interfaz gráfica completa con Swing:
 - VentanaPrincipal (JFrame) con menú de navegación.
 - PanelPacientes: formulario de alta/edición, tabla (JTable) de listado, botones CRUD.
 - PanelOdontologos: formulario de alta/edición, tabla de listado, botones CRUD.
 - PanelTurnos: formulario de reserva con selectores de paciente y odontólogo, tabla de turnos, filtros por fecha/estado.
 - PanelBusquedas: búsqueda por DNI, por matrícula, turnos por fecha.
- Manejo de eventos: ActionListener en botones, MouseListener en tablas (doble clic para editar), KeyListener para validación en tiempo real (opcional).
- Validación visual de formularios: campos obligatorios resaltados, mensajes de error en JOptionPane o JLabel.
- (Opcional) Uso de hilos (Thread/Runnable) para tareas asíncronas: recordatorios de turnos próximos, búsquedas que no bloqueen la interfaz, carga de datos en segundo plano.
- Integración completa: GUI → Servicio → Repositorio → Archivos. Sistema funcional end-to-end.
- Sistema funcional completo: desde la GUI se pueden realizar todas las operaciones del sistema con datos que persisten entre ejecuciones.

Conceptos evaluados

GUI Swing (JFrame, JPanel, JTable, JButton, JTextField, JComboBox, JOptionPane, layouts), eventos (ActionListener, MouseListener), hilos (Thread, Runnable), integración de capas.

Formato de entrega

- Código fuente completo del proyecto Java.
- Archivo JAR ejecutable (opcional pero recomendado).
- Manual de usuario breve (capturas de pantalla de la GUI con descripción de funcionalidades).
- Archivo PDF con todos los diagramas UML finales.
- Informe final (4–5 páginas): arquitectura final, patrones aplicados, decisiones de diseño de la GUI, reflexión sobre el proceso de desarrollo, lecciones aprendidas.

10. Criterios de Evaluación

Cada entrega se evalúa según los siguientes criterios, con ponderaciones que reflejan la importancia relativa de cada aspecto en la formación profesional del desarrollador.

10.1 Ponderación general

Criterio	Ponderación	Descripción
Funcionalidad	40%	El sistema hace lo que debe hacer: todas las operaciones requeridas funcionan correctamente, sin errores de ejecución.
Diseño orientado a objetos	30%	Aplicación correcta de POO (encapsulamiento, herencia, polimorfismo), patrones GRASP, principios SOLID, relaciones bien modeladas.
Calidad de código	20%	Código limpio, bien estructurado, con nombres significativos, sin duplicación, documentado con Javadoc donde corresponda.
Documentación	10%	Diagramas UML correctos y actualizados, informes con reflexión técnica, comentarios pertinentes en el código.

10.2 Criterios específicos por entrega

Entrega 1:

- Clases de dominio correctamente implementadas con encapsulamiento.
- Relaciones entre clases bien modeladas (composición Paciente–Domicilio).
- Diagrama UML coherente con el código.
- Código compilable y ejecutable.

Entrega 2:

- Arquitectura en capas visible y coherente (dominio, repositorio, servicio, presentación).
- Patrones GRASP identificados y aplicados correctamente (con justificación).
- Principios SOLID demostrados (especialmente SRP y OCP).
- CRUD funcional desde consola.

Entrega 3:

- Excepciones personalizadas bien diseñadas y utilizadas en todos los servicios.
- Persistencia funcional: datos se guardan y cargan correctamente.
- Uso efectivo de colecciones para búsquedas y filtros.
- Diagramas de secuencia correctos y coherentes con el código.

Entrega 4:

- GUI funcional e intuitiva que permite realizar todas las operaciones.
- Manejo de eventos correcto (botones, tablas, formularios).
- Integración completa de todas las capas (GUI → Servicio → Repositorio → Archivos).
- Sistema end-to-end funcional y demostrable.
- (Bonus) Uso de hilos para tareas asíncronas.

11. Tecnologías y Herramientas

11.1 Tecnologías permitidas

Categoría	Tecnología	Detalle
Lenguaje	Java SE 8 o superior	Se recomienda Java 11 o 17 LTS. Se utilizará exclusivamente Java Standard Edition.
Bibliotecas	java.util.*	Colecciones: ArrayList, HashMap, LinkedList, TreeMap, HashSet, etc.
	java.io.*	Lectura/escritura de archivos: BufferedReader, BufferedWriter, ObjectOutputStream, etc.
	java.time.*	Manejo de fechas y horas: LocalDate, LocalTime, LocalDateTime.
	javax.swing.*	Componentes gráficos: JFrame, JPanel, JTable, JButton, JTextField, etc.
	java.awt.*	Layouts, eventos, gráficos: BorderLayout, GridLayout, ActionListener, etc.
IDE	A elección	Eclipse, IntelliJ IDEA, NetBeans, VS Code con extensión Java.
UML	A elección	StarUML, Draw.io (diagrams.net), Lucidchart, Visual Paradigm.
Versionado	Git (recomendado)	GitHub, GitLab o Bitbucket para repositorio remoto.

11.2 Tecnologías NO permitidas

El objetivo del proyecto es dominar los fundamentos de Java OO puro. Por lo tanto, no se permite el uso de las siguientes tecnologías en ninguna entrega:

- Spring Boot, Spring Framework, Spring Data, Spring MVC ni ningún módulo de Spring.
- Hibernate, JPA ni ningún ORM (Object-Relational Mapping).
- Bases de datos relacionales (H2, MySQL, PostgreSQL, etc.). La persistencia es exclusivamente en archivos.
- APIs REST, controladores HTTP, Servlets.
- HTML, CSS, JavaScript, Bootstrap, Thymeleaf ni ninguna tecnología web.
- DTOs (Data Transfer Objects) como patrón de transferencia web (las entidades se usan directamente).
- Anotaciones de Spring (@Autowired, @Service, @Repository, @Controller, @RestController, etc.).

- Bean Validation (@Valid, @NotNull, @Size, etc.).
- Maven/Gradle como gestores de dependencias externas (pueden usarse para compilación, pero sin dependencias de terceros).
- Cualquier biblioteca o framework externo a Java SE.

12. Recomendaciones y Buenas Prácticas

Las siguientes recomendaciones están inspiradas en prácticas profesionales de la industria del software. Adoptarlas desde el inicio de la formación genera hábitos que marcan la diferencia en la carrera profesional.

12.1 Nomenclatura y convenciones Java

- **Clases e interfaces:** PascalCase (ej: ServicioPaciente, IRepository, PacienteNoEncontradoException).
- **Métodos y variables:** camelCase (ej: buscarPorDni(), fechaIngreso, listarTodos()).
- **Constantes:** UPPER_SNAKE_CASE (ej: ESTADO_PENDIENTE, MAX_TURNOS_POR_DIA).
- **Paquetes:** minúsculas sin guiones (ej: com.clinica.sonrisafeliz.servicio).

12.2 Código limpio (Clean Code)

- Nombres significativos: las clases, métodos y variables deben revelar su intención. Evitar nombres genéricos como 'datos', 'info', 'temp'.
- Métodos cortos y con una sola responsabilidad: si un método hace demasiadas cosas, dividirlo.
- Evitar duplicación de código (principio DRY – Don't Repeat Yourself).
- Eliminar código muerto (comentado o inalcanzable) antes de entregar.
- Documentar con Javadoc las clases públicas y los métodos principales.

12.3 Diseño orientado a objetos

- Aplicar encapsulamiento siempre: atributos privados, acceso mediante métodos públicos.
- Preferir composición sobre herencia cuando no exista una relación "es-un" genuina.
- Programar contra interfaces, no contra implementaciones (ej: IRepository<T> en lugar de RepositorioPaciente directamente).
- Aplicar el principio de mínimo conocimiento: cada clase solo debe conocer lo estrictamente necesario.
- Buscar alta cohesión (cada clase hace una cosa bien) y bajo acoplamiento (mínimas dependencias entre clases).

12.4 Control de versiones

- Hacer commits frecuentes con mensajes descriptivos (ej: "Agrega ServicioPaciente con CRUD completo").
- No commitar código que no compila.
- Utilizar un archivo .gitignore para excluir archivos innecesarios (.class, .idea, *.iml, etc.).
- Si trabajan en equipo, utilizar ramas (branches) para funcionalidades y hacer merge al completar.

12.5 Proceso de desarrollo

- Desarrollar de forma incremental: no intentar hacer todo de una vez. Implementar, probar, avanzar.

- Probar cada funcionalidad a medida que se implementa. No dejar las pruebas para el final.
- Leer los mensajes de error de Java con atención: la stack trace indica exactamente dónde y por qué falla el programa.
- Consultar dudas en clase, en horario de consulta o por los canales habilitados. No avanzar con incertidumbre.
- Revisar el código de los compañeros (code review informal) para aprender y mejorar mutuamente.

12.6 Relevancia profesional

Cada concepto que se aplica en este proyecto tiene un correlato directo en la industria del software:

- **POO y patrones de diseño:** son la base de cualquier desarrollo empresarial en Java, C#, Python, etc.
- **Arquitectura en capas:** es el estándar en aplicaciones empresariales; frameworks como Spring la implementan con convenciones, pero el concepto subyacente es el que se aprende aquí.
- **Excepciones personalizadas:** todo sistema robusto las utiliza para comunicar errores de negocio de forma clara.
- **Colecciones:** se usan en absolutamente todo: desde APIs hasta procesamiento de datos y machine learning.
- **Control de versiones (Git):** es una habilidad no negociable en cualquier equipo de desarrollo.
- **GUI:** aunque hoy predominan las interfaces web, comprender eventos, hilos y componentes gráficos es fundamental para cualquier tipo de interfaz.

13. Metodología de Trabajo

El proyecto sigue un enfoque iterativo e incremental inspirado en metodologías ágiles, adaptado al contexto académico. Cada entrega es un incremento funcional sobre la anterior.

13.1 Roles

- Docente (Product Owner y Arquitecto): define los requerimientos, prioriza entregas, establece estándares técnicos, guía decisiones de diseño.
- Estudiantes (Equipo de desarrollo): implementan las funcionalidades, diseñan diagramas UML, documentan, prueban y presentan cada entrega.

13.2 Dinámica de entregas

- Cada entrega tiene una fecha límite inamovible alineada con el cronograma de clases.
- Las entregas son acumulativas: la Entrega 2 incluye todo lo de la Entrega 1 mejorado, y así sucesivamente.
- Se espera una demo funcional en cada entrega: el sistema debe ejecutarse y mostrar las funcionalidades requeridas.
- Los informes técnicos deben reflejar reflexión sobre las decisiones de diseño, no solo describir qué se hizo.

13.3 Definition of Done (DoD)

Una entrega se considera completa cuando cumple todos los siguientes criterios:

- El código compila y ejecuta sin errores.
- Todas las funcionalidades requeridas para la entrega están implementadas y funcionan.
- El código sigue las convenciones de nomenclatura Java.
- Los diagramas UML están actualizados y son coherentes con el código.
- El informe técnico está completo y refleja las decisiones de diseño.
- Se puede hacer una demostración funcional en clase.

14. Anexos

14.1 Glosario de términos

Término	Definición
CRUD	Create, Read, Update, Delete – las cuatro operaciones básicas de gestión de datos.
DTO	Data Transfer Object – patrón para transferir datos entre capas (NO aplica en este proyecto).
GRASP	General Responsibility Assignment Software Patterns – patrones de asignación de responsabilidades en diseño OO.
GUI	Graphical User Interface – interfaz gráfica de usuario.
Javadoc	Herramienta de documentación de Java que genera documentación HTML a partir de comentarios en el código.
OCP	Open/Closed Principle – principio SOLID: abierto a extensión, cerrado a modificación.
POJO	Plain Old Java Object – clase Java simple sin dependencias de frameworks.
Serialización	Proceso de convertir un objeto en una secuencia de bytes para almacenarlo o transmitirlo.
SOLID	Cinco principios de diseño OO: SRP, OCP, LSP, ISP, DIP.
SRP	Single Responsibility Principle – cada clase debe tener una única razón para cambiar.
Swing	Biblioteca de Java para crear interfaces gráficas de escritorio.
UML	Unified Modeling Language – lenguaje estándar para modelado de sistemas de software.

14.2 Referencias bibliográficas

- Sierra, K. & Bates, B. – "Head First Java" (O'Reilly). Referencia amigable para aprender Java OO.
- Bloch, J. – "Effective Java" (Addison-Wesley). Buenas prácticas avanzadas en Java.
- Martin, R.C. – "Clean Code" (Prentice Hall). Principios de código limpio y profesional.
- Gamma, E. et al. – "Design Patterns" (Addison-Wesley). Catálogo clásico de patrones de diseño.
- Larman, C. – "Applying UML and Patterns" (Prentice Hall). Referencia fundamental para GRASP y UML.
- Eckel, B. – "Thinking in Java" (Prentice Hall). Comprensión profunda del lenguaje Java.
- Oracle – Documentación oficial de Java SE: <https://docs.oracle.com/javase/>
- Oracle – Tutorial de Swing: <https://docs.oracle.com/javase/tutorial/uiswing/>

14.3 Ejemplo de código – Clase Paciente (referencia)

```
public class Paciente {
    private Long id;
    private String nombre;
    private String apellido;
    private String dni;
    private String email;
    private LocalDate fechaIngreso;
    private Domicilio domicilio;

    // Constructor vacío
    public Paciente() {}

    // Constructor parametrizado
    public Paciente(String nombre, String apellido, String dni,
                     String email, Domicilio domicilio) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.dni = dni;
        this.email = email;
        this.fechaIngreso = LocalDate.now();
        this.domicilio = domicilio;
    }

    // Getters y Setters (omitidos por brevedad)

    public String getNombreCompleto() {
        return nombre + " " + apellido;
    }

    @Override
    public String toString() {
        return "id=" + id +
            ", nombre='" + getNombreCompleto() + '\'' +
            ", dni='" + dni + '\'' +
            ", domicilio=" + domicilio +
            '}' ;
    }
}
```

Mensaje Final

Este proyecto es mucho más que una práctica de programación: es una experiencia de diseño, construcción y entrega de valor. La clínica "Sonrisa Feliz" es el contexto; la ingeniería de software orientada a objetos, el camino; cada entrega, un hito profesional.

No busquen la solución perfecta desde el inicio. Busquen avances reales y visibles en cada etapa. Cada decisión técnica que tomen —desde cómo nombrar una clase hasta cómo organizar los paquetes— es una oportunidad de aprender a pensar como ingenieros de software.

Al final del recorrido, no solo tendrán un sistema funcional: tendrán la experiencia de haberlo diseñado, construido, probado y presentado. Eso es lo que vale.

— Cátedra de Paradigma Orientado a Objetos